

CSE 585: Advanced Scalable Systems

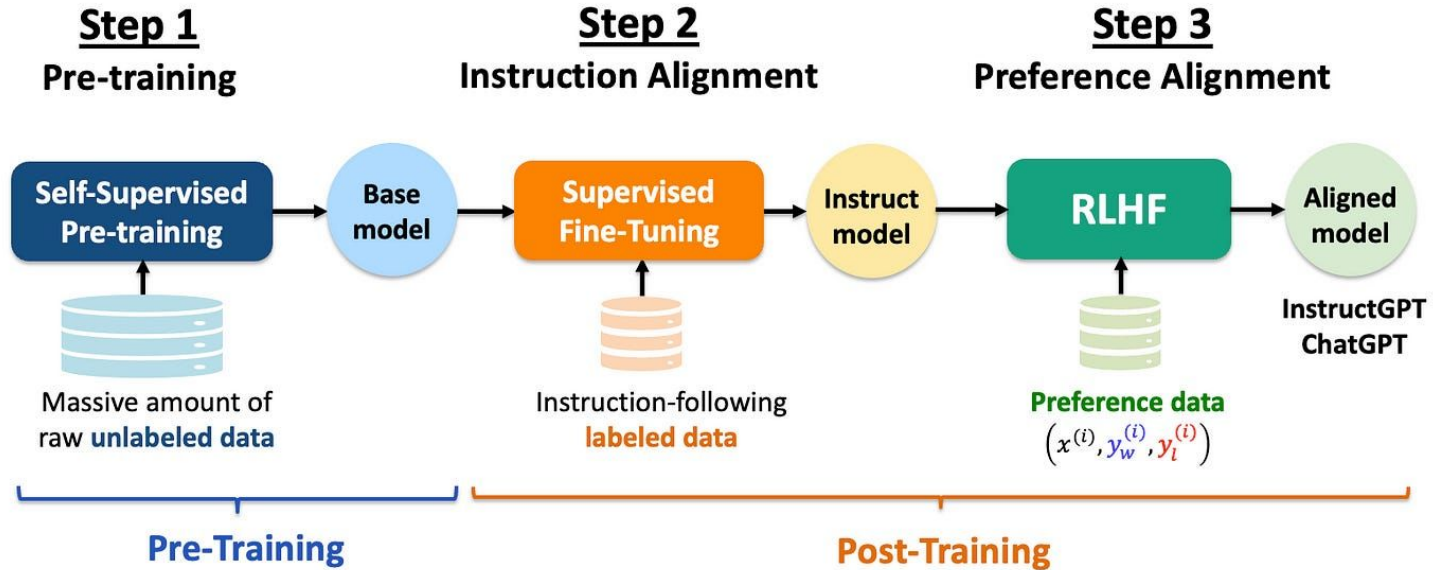
Post-Training

Aiden Dsouza
Arshdeep Singh
Samir Khanna
Sri Satya Vinay Venneti

: aidenrds@umich.edu
: arshsb@umich.edu
: samirkh@umich.edu
: srisatya@umich.edu

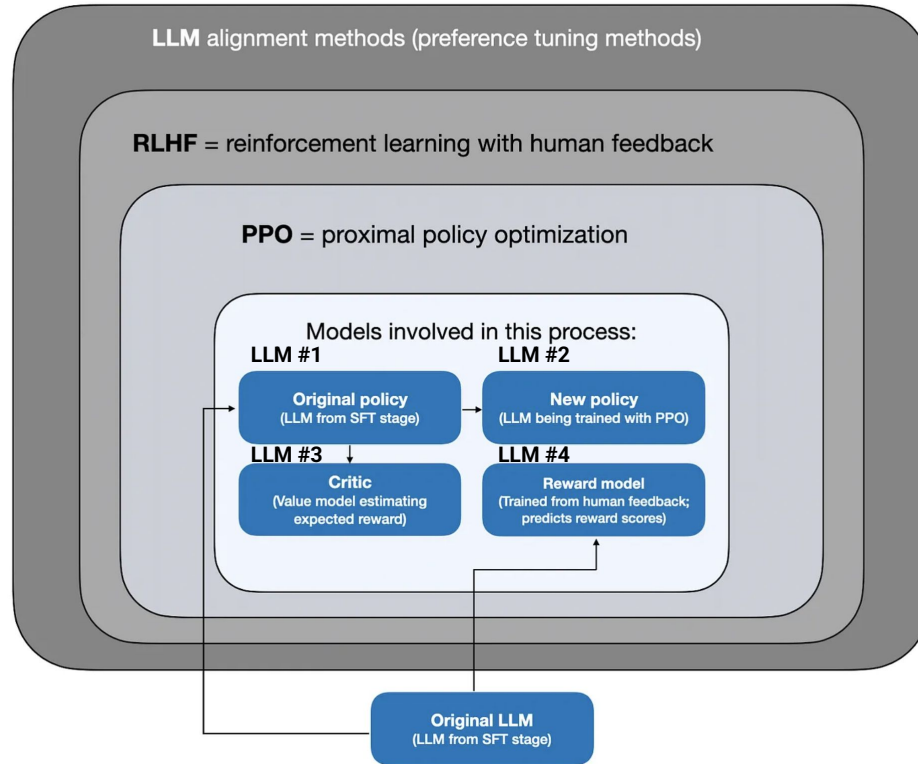
HybridFlow: A Flexible and Efficient RLHF Framework

Background: LLM Training Process



Source: Adapted from Hosni, [Visual Guide to LLM Preference Tuning With RLHF & PPO](#) (2025).

Background: LLM RL Jargon



Source: Sebastian Raschka, [*The State of Reinforcement Learning for LLM Reasoning: Understanding GRPO and New Insights from Reasoning Model Papers*](#) (2025).

Phases in RLHF

Training

1 forward, 1 backward,
1 weight update

Inference (Prefill)

1 forward

Generation (Decode)

Auto-regressive forward pass

In one iteration of RLHF using PPO

Needs to be stored

Actor

Generation, Training

Model weights, gradients,
parameters, KV cache

Critic

Inference, Training

Model weights, gradients,
parameters

Reward, Reference Policy

Inference

Model weights

PPO Algorithm

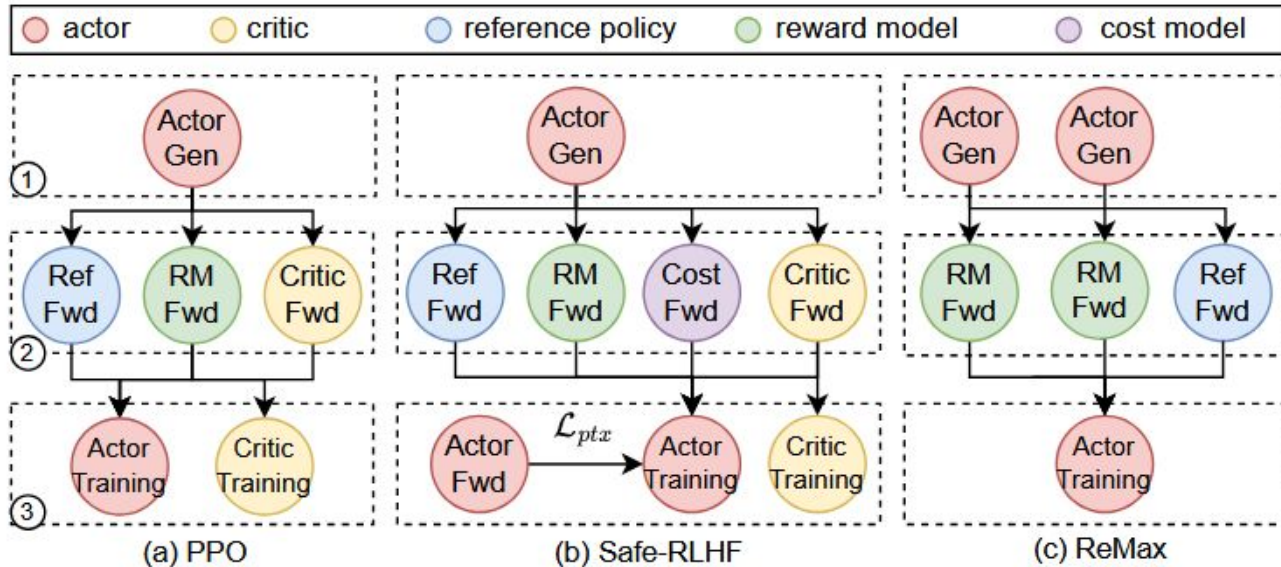


Figure 1. Dataflow graph of 3 RLHF algorithms [19, 43, 55]. Stage ①, ②, ③ represent Generation, Preparation, and Training, respectively.

Problem #1: RLHF Has a Complex Dataflow

Distributed Nodes

Each LLM requires specific parallel computing strategies (tensor, pipeline, data).

Complex Edges

Data transfer becomes heavy, many-to-many multicasting across model partitions.

The Need: RLHF's extreme demands require highly flexible and efficient execution systems

Problem #2: Existing Single-Controller and Multi-Controller Systems Become Inefficient

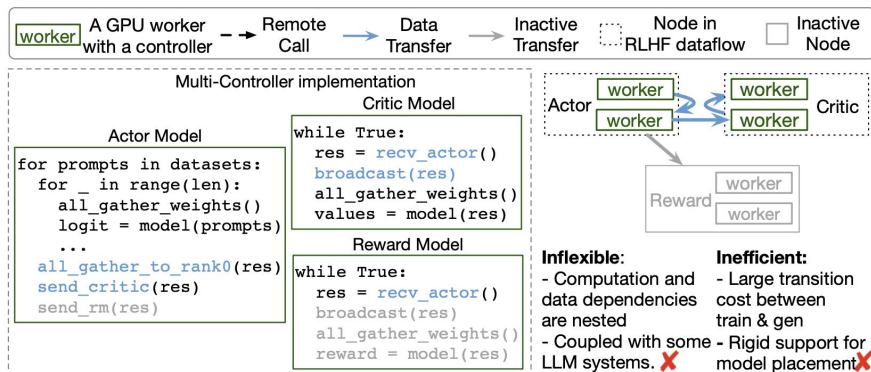
Single controller

- **Centralized control**
- **Small-model design**
- **Dispatch bottleneck**

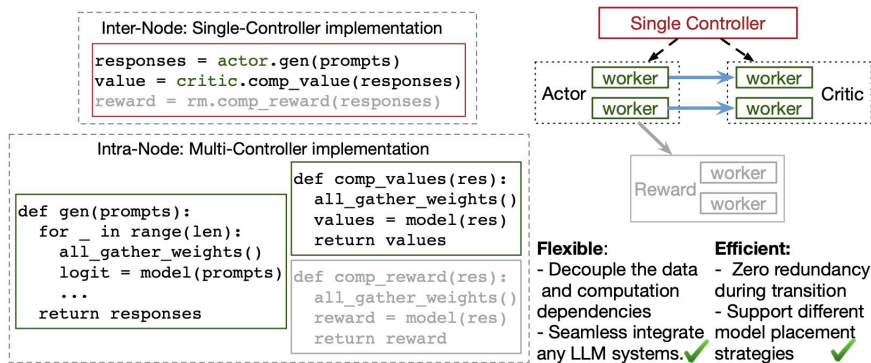
Multi-controller

- **Decentralized control**
- **Local view only**
- **Tight coupling**

Problem #2 (Continued)



(a) Existing RLHF frameworks



(b) HybridFlow

Problem #3: Misallocation of Resources for Actor Model During Training and Generation

Different demands

The two phases in one RLHF iteration have different compute and memory demands

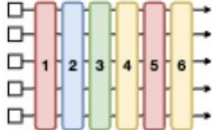
Different parallelism

So each phase can use its own parallelism architecture

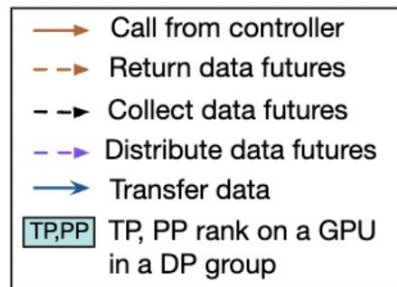
Wasted resources

Existing systems keep two completely separate sets of GPUs, wasting money and resources

What If RLHF Could Be Flexible Without Sacrificing Efficiency?

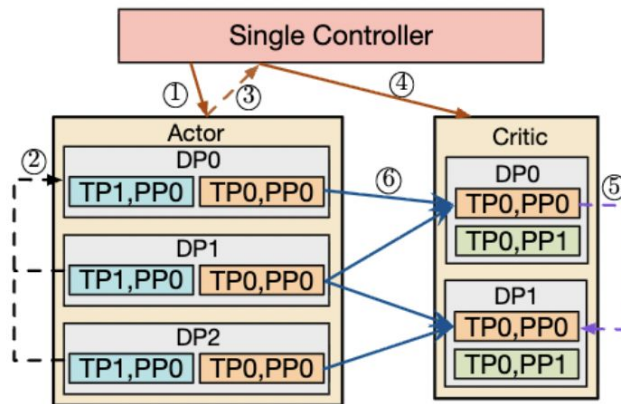
RLHF system	DeepSpeed-Chat	OpenRLHF	NeMo-Aligner	HybridFlow
Parallelism	Training: ZeRO Generation: TP	Training: ZeRO Generation: TP	3D Parallelism for both training and generation	Training: 3D, ZeRO, FSDP Generation: 3D Parallelism
Actor weights in training & generation	Model resharding from ZeRO to TP	Using two copies of actor weights for the two stages	Using identical model partition in two stages (shared weights)	Zero-redundancy model resharding
Model Placement	Colocate all models on the same set of devices	Each model placed on separate devices	Actor/Ref colocated on some GPUs Critic/RM colocated on other GPUs	Support various model placement
Execution Pattern Actor Critic Reward model Reference Policy GPU Process				Support various execution patterns

Contribution 1: Hybrid Controller API Flexibility



Actor (p, t, d) = (1, 2, 3)
Critic (p, t, d) = (2, 1, 2)

(b) Data resharding and asynchronous execution



Single Controller (Inter)

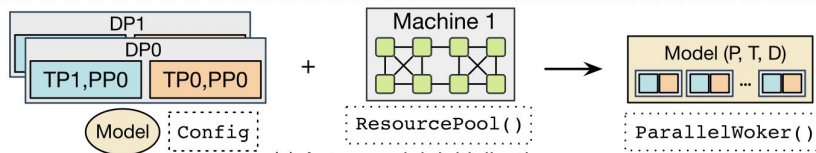
- RLHF workflow
- Model-model communication
- Data dependencies
- Resource coordination

Multi-Controller (Intra)

- Distributed computation
- GPU-level execution
- Parallelism within each model

Hybrid Programming Model Structure

```
class ActorWorker(3DParallelWorker):
    # An example of distributed computation function
    @register(transfer_mode=3D_PROTO)
    def update_actor(self, prompts: DataProto):
        ...
# Allocate devices for a ResourcePool
resource_pool = ResourcePool([n_gpus_per_machine] * n_machines)
# Map the model to allocated devices and init model
actor_model = ActorWorker(actor_config, resource_pool)
```



(a) Actor model initialization

```
# Initialize cost model by reusing the RewardWorker
cost = RewardWorker(cost_config, resource_pool)
... # omit other models initialization
algo_type = "Safe-RLHF" # specify different RLHF numerical computation.
# Examples of PPO and Safe-RLHF
for (prompts, pretrain_batch) in dataloader:
    # Stage 1: Generate responses
    batch = actor.generate_sequences(prompts)
    batch = actor.generate_sequences(prompts, do_sample=False)
    # Stage 2: Prepare experience
    batch = critic.compute_values(batch)
    batch = reference.compute_log_prob(batch)
    batch = reward.compute_reward(batch)
    batch = cost.compute_cost(batch)
    batch = compute_advantages(batch, algo_type)
    # Stage 3: Actor and critic training
    critic_metrics = critic.update_critic(batch, loss_func=algo_type)
    pretrain_loss = actor.compute_loss(pretrain_batch)
    batch["pretrain_loss"] = pretrain_loss
    actor_metrics = actor.update_actor(batch, loss_func=algo_type)
```

[] is added for ReMax
 [] Not necessary in ReMax
 [] is added for Safe-RLHF

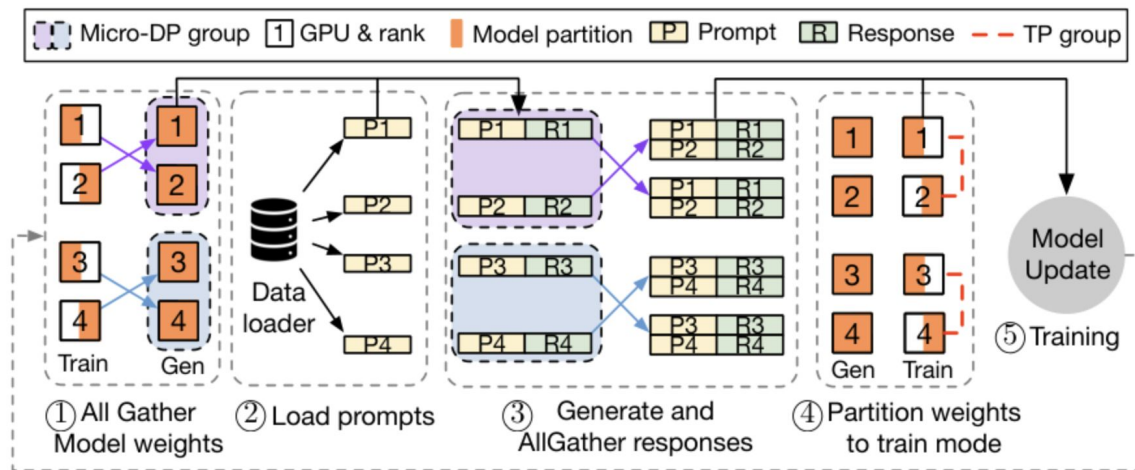
When switching phases, an actor must currently undergo a memory-intensive resharding process or migrate to a new set of otherwise idle GPUs. Can we optimize this?

Can we reshard it in a way that reduces memory overhead?

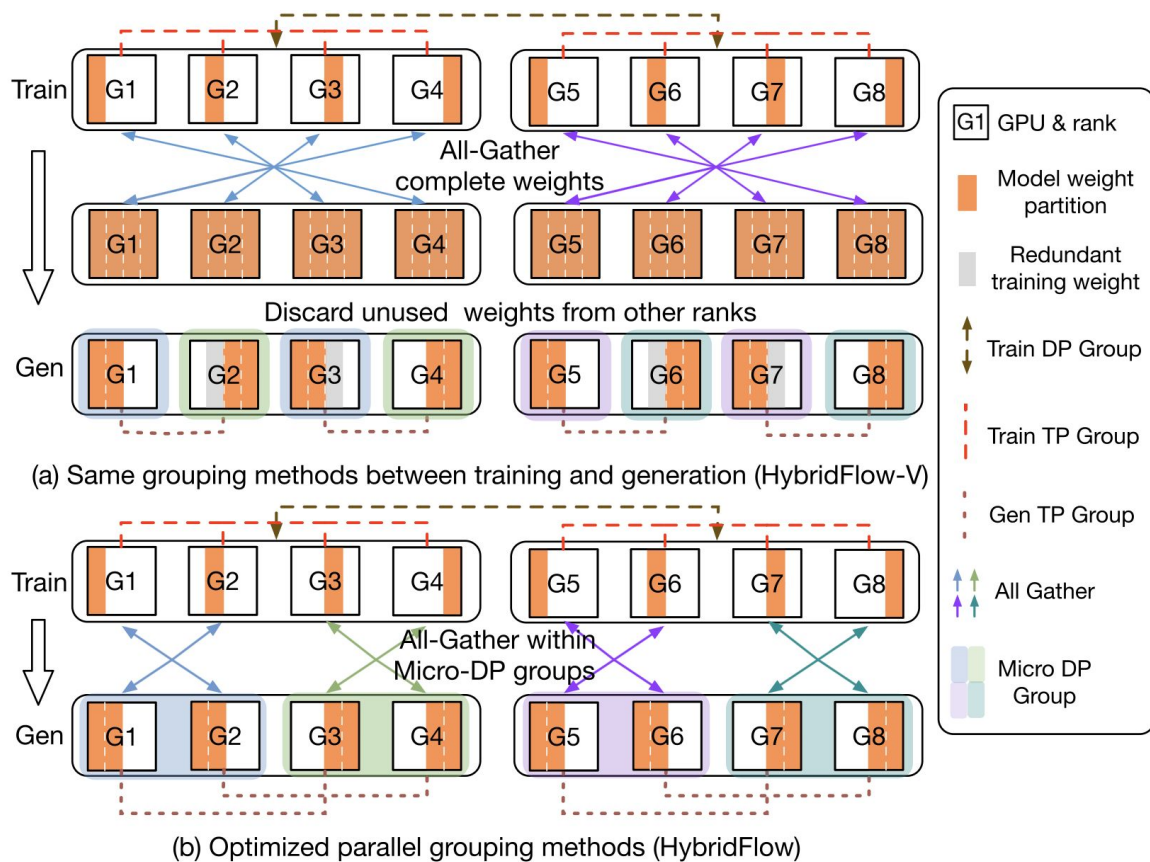
Contribution 2: 3D-HybridEngine: How it Works?

One RLHF iteration

1. All-gather weights in micro-DP groups
2. Load prompts into each replica
3. Generate, then all-gather responses
4. Repartition weights for training
5. Compute loss, update actor weights



Training weights are reused for generation, so nothing is stored twice.



Contribution #3: Auto Device Mapping: Placement and Parallelism

Why it's needed	Sharing GPUs avoids idle time, separate GPUs allow parallel runs. The best mix changes with model size and cluster size.
Goal & constraints	Goal: minimize latency of one RLHF iteration Constraints: memory per GPU (no OOM), total GPU count
Approach	For each colocation plan, start from the minimum GPUs that fit in memory, try all allocations, and keep the lowest simulated latency.
Best possible?	Yes, within its model: the search is exhaustive. Real-world quality depends on how accurate the simulator is.
Complexity	Worst case: $O((N-1)! / ((k-1)!(N-k)!))$ for N GPUs, k models

Overall Framework

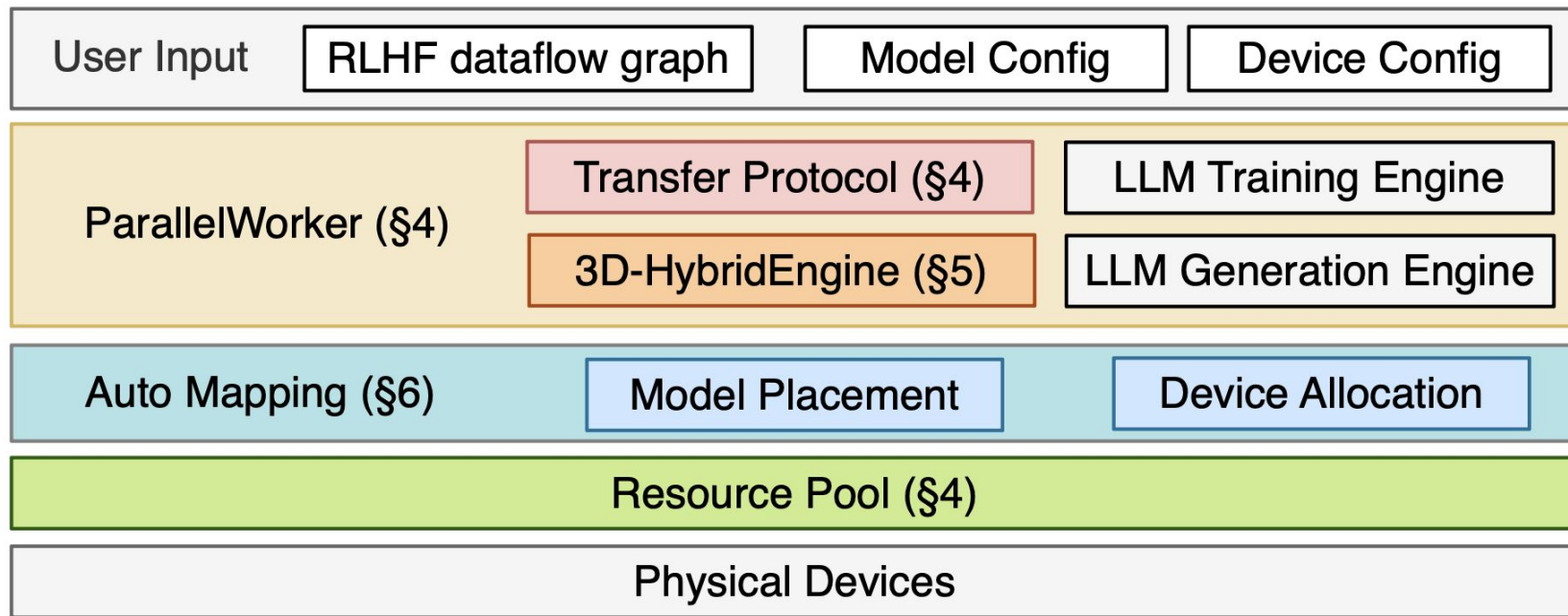


Figure 4. Architecture of HybridFlow.

Evaluation - What is Being Tested?

E2E Performance

Does throughput increase?

Scalability

How does it perform when workload and GPU cluster scale?

Automatic Device Mapping

Does automatic allocation outperform fixed strategies?

3D-HybridEngine

Does 3D-HybridEngine actually reduce transition overhead?

Evaluation Setup

Testbed

16 machines × 8 GPUs
128 NVIDIA A100 80GB
GPUs
600 GB/s NVLink
200 Gbps inter-machine

Models

Llama 7B
Llama 13B
Llama 34B
Llama 70B

RLHF workloads

PPO
ReMax
Safe-RLHF

Baselines

DeepSpeed-Chat
OpenRLHF
NeMo-Aligner

Throughput: Total prompt + response tokens processed per second per RLHF iteration.

This setup tests whether the system-level ideas hold across both workload scale (7B→70B) and distributed resource scale (up to 128 GPUs).

Main Result: HybridFlow Improves RLHF Throughput

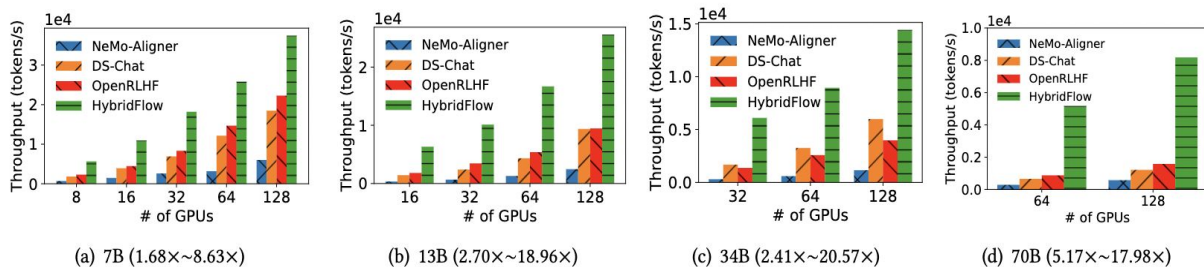


Figure 9. PPO throughput. Numbers in parentheses are HybridFlow speedups compared with baselines.

1.53x–20.57x

Overall throughput improvement reported by the paper

9.64x

highest average speedup on 70B

66.8%

strong-scaling efficiency

Baseline Context

For PPO, average speedups are 3.67x vs. DeepSpeed-Chat, 3.25x vs. OpenRLHF, and 12.52x vs. NeMo-Aligner.

Why The Gains Happen? Orchestration Overhead Matters

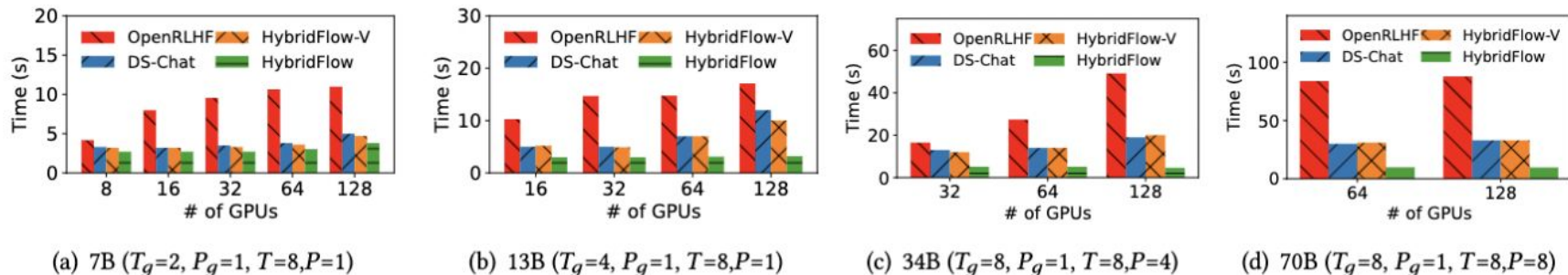
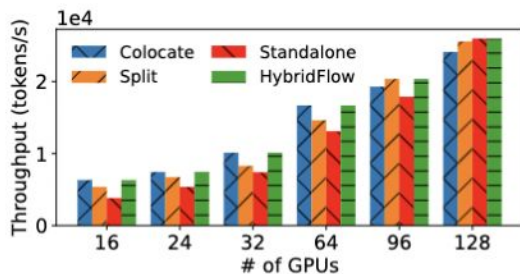


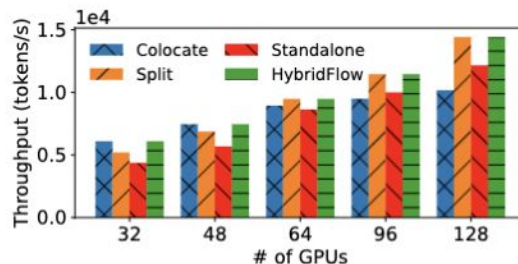
Figure 14. Transition time between actor training and generation.

Cheaper actor transitions: 3D-HybridEngine reshards actor parameters without duplicating model memory.

Optimal Placement Changes As Workload/GPU Scale



(a) 13B



(b) 34B

Figure 12. Throughput of HybridFlow under different placements

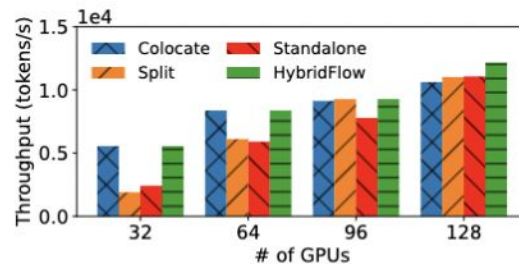


Figure 13. Placement comparison under 13B actor and reference policy & 70B critic and reward model.

16–64 GPUs: colocation often wins

96–128 GPUs: split / standalone often wins

Discussion & Limitations

Discussion

Fault tolerance & recovery

Checkpointing, NCCL error handling, and checksum validation support recovery when workers fail.

Resource multiplexing

Time-sharing lets models share GPUs, sequential execution avoids contention and OOM failures.

Beyond RLHF

The abstraction can extend to programmatic rewards, such as sandbox or mathematical verification.

Limitations

GPU scheduling

Current time-sharing is coarse-grained, so models execute sequentially rather than concurrently.

Heterogeneous resources

The system is designed primarily for homogeneous GPU nodes; finer-grained offloading is a future direction.

Control / mapping scalability

Auto-mapping runtime grows with model and cluster size. caching is used to bound search time.

ROLLART: Disaggregated Multi-Task Agentic RL Training at Scale

Scope: RollArt vs. HybridFlow

Both target RL post-training of LLMs — but for **fundamentally different workloads**.

HybridFlow

Conventional RLHF

- **No environment:** one response per prompt
- **GPU-only workload:**
actor/critic/ref/reward are all GPU passes
- **System problem:** schedule a GPU-only model dataflow

RollArt

Multi-Turn Agentic RL

- **Real environment:** agent acts & observes over many turns
- **Mixed workload:** GPU gen/training of agents that run jobs on the CPU (env)
- **System problem:** coordinate heterogeneous compute around the model

In short: HybridFlow optimizes **how GPUs talk to each other**; RollArt coordinates **GPUs + CPUs + serverless platforms around a real environment**.

Background

Agentic RL Training Pipeline

Rollout: agent $\leftrightarrow$ environments $\rightarrow$ **trajectories**



Reward: scores trajectories $\rightarrow$ **scalar reward**



Training: PPO/GRPO uses trajectories + rewards $\rightarrow$ **updates LLM**

LLM Inference Phases

Prefill: full prompt in parallel, builds KV cache $\rightarrow$ **compute-bound** – sets Time to First Token (**TTFT**)



Decode: one token at a time from the KV cache $\rightarrow$ **bandwidth-bound** – sets Inter-Token Latency (**ITL**)

Hardware specifications of GPUs

Compute capability (TFLOPS)

How fast the GPU can perform mathematical operations.

$\rightarrow$ **H800 GPUs are better**

Memory bandwidth (HBM bandwidth)

How fast data moves between GPU memory and compute units.

$\rightarrow$ **H20 GPUs are better**

The Motivation: Heterogeneity & Resource Mismatches

Heterogeneous workload — *every pipeline stage needs different resources*



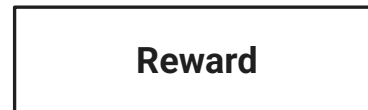
compute-bound



bandwidth-bound



CPU / network



serverless / bursty

Heterogeneous environments — *different compute profiles during inference*

Prefill-Heavy

Tasks: SWE-bench, WebShop, Frozen Lake
(multi-turn, long context)

→ **Compute-optimized GPUs (H800s)**

Decode-Heavy

Tasks: GEM-Math, GEM-Game (few turns, long reasoning)

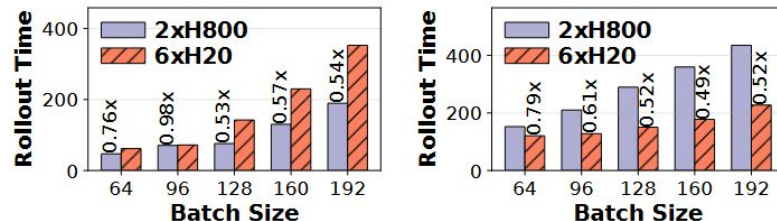
→ **Bandwidth-optimized GPUs (H20s)**

Key limitation: Today's systems colocate stages on one GPU or **statically split** rollout / training — this can't adapt to shifting agentic workloads.

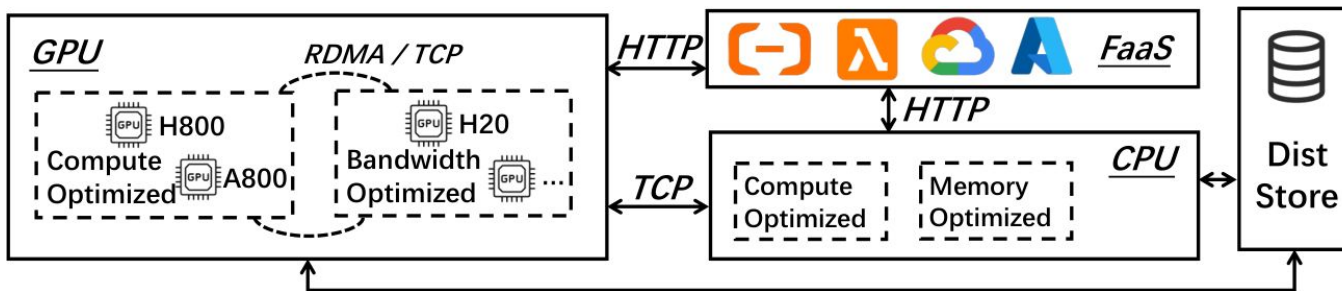
Design Pillar 1: Hardware-Affinity Workload Mapping

Existing systems bind all phases to a single GPU type. Agentic RL workloads are heterogeneous. No single GPU fits all. **Can hardware be matched to each workload's real compute profile instead?**

- **Training** → H800 (high compute)
- **Generation** → H20 default (high memory bandwidth);
prefill-heavy → H800
- **Environments** → CPU clusters + Kubernetes
- **Reward** → serverless (bursty demand)



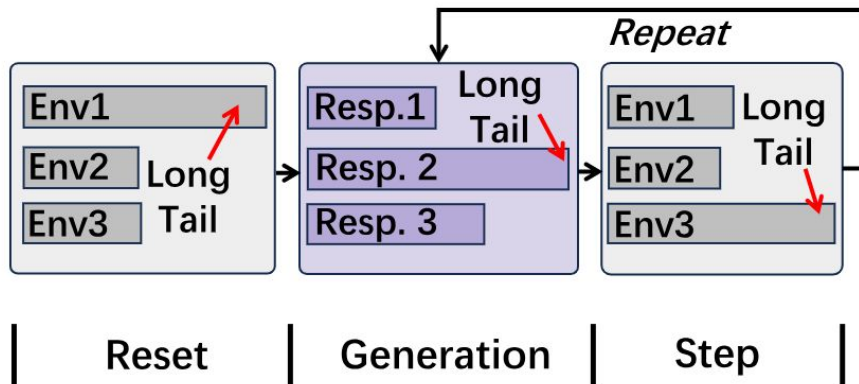
(a) FrozenLake [Prefill-Heavy]. (b) GEM-Math [Decode-Heavy].



Design Pillar 2: Trajectory-Level Asynchronous Rollout

*Batched implementation forces fast environments to wait for the slowest straggler. **Can this be improved?***

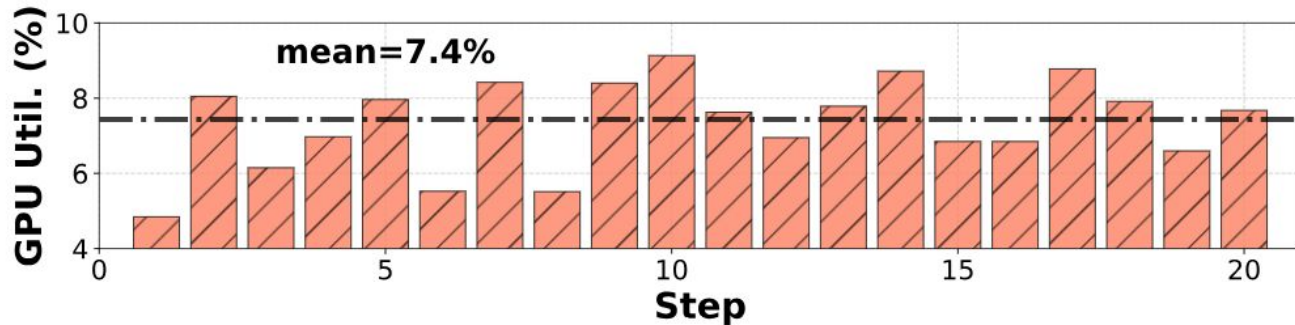
- **Environment execution** has extreme long-tail latency (due to network/ compute/ I/O contention)
- **Batch-level sync:** fast environments wait for stragglers
- **Solution:** process each trajectory independently



Design Pillar 3: Serverless Reward Offloading

*Dedicating GPUs for reward computation leaves them idle most of the time. **Can this be improved?***

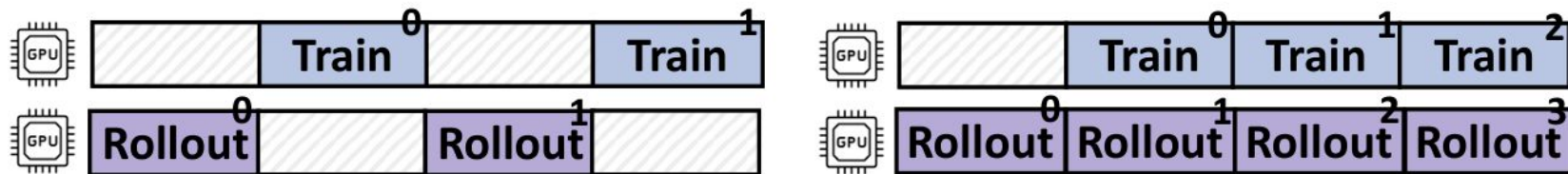
- **Rewards** are stateless + bursty
- **Dedicated reward GPUs** → only 7.4% utilization
- **Offload to elastic serverless** → autoscaling, no idle GPUs



Design Pillar 4: Bounded-Staleness Asynchronous Training

*Weight synchronization is communication-heavy, and synchronous training blocks rollout until the latest weights arrive, leaving GPUs idle during every multi-GB transfer. **Can this be improved?***

- **Synchronous:** rollout waits for latest weights → idle GPUs / dependency bubbles
- **Asynchronous:** rollout & training run concurrently → higher utilization, slight staleness
- **Bounded staleness α** eliminates bubbles while preserving performance



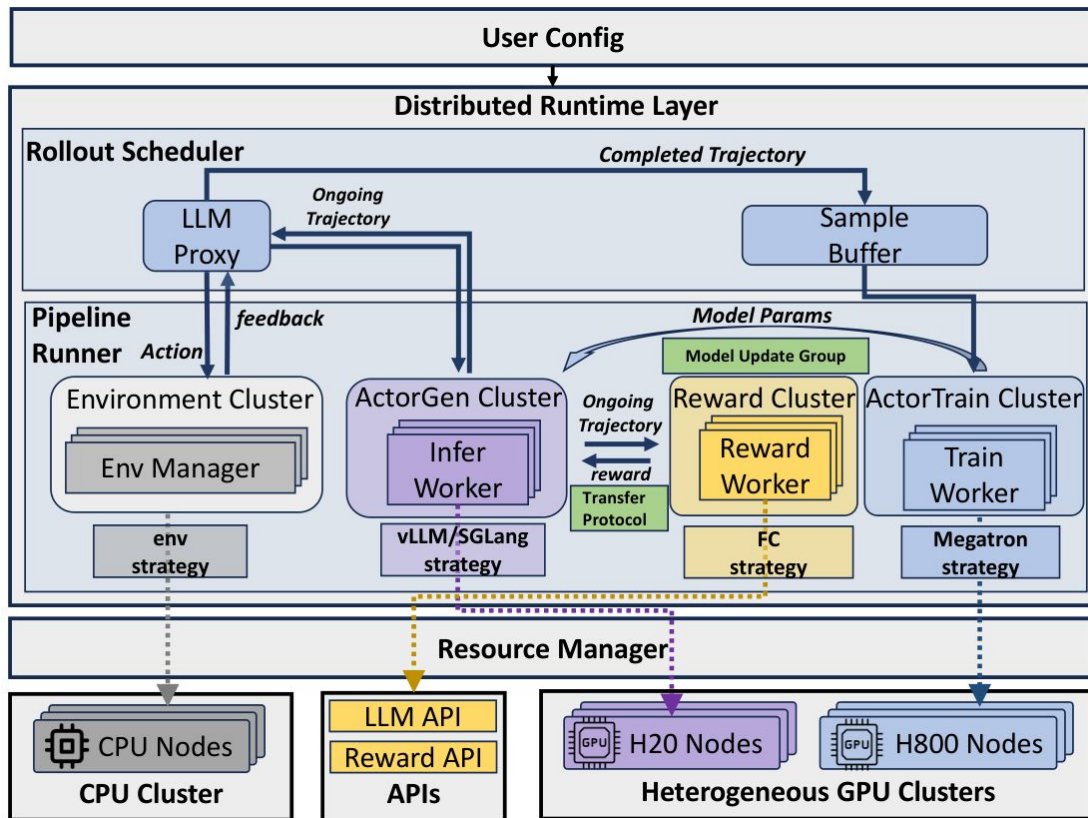
Architecture

3 Planes

- **Resource plane:** Hardware allocation
- **Data plane:** Executes RL job on role-specific clusters
- **Control plane:** Interaction among clusters via **LLMProxy** and **SampleBuffer**

Worker / Cluster Abstractions

- **Worker:** basic execution unit
- **Cluster:** manages group of Workers



Worker Implementation Snippet

```
1 import rollart.distributed as rdist
2 from rdist.worker import ActorTrainCls, ActorGenCls,
  RewardCls
3 from rdist import ResourceManager as RM
4
5 # 1. Single Controller Example
6 class MyActorTrain(ActorTrainCls):
7     @rdist.register(mode="execute_all")
8     def compute_gradients(self, input_tensor):
9         ...
10
11 # 2. Define actor_gen on heterogeneous GPUs
12 # 2.1 heterogeneous GPU allocation.
13 gen_rm=RM( {"H800": list(range(0, 8))},
14            {"H20", list(range(8, 32))})
15 # 2.2 hardware affinity mapping
16 class HeteroActorGen(ActorGenCls):
17     @rdist.hw_mapping(
18         hw_affinity={"FrozenLake": "H800", "default": "H20"}
19     )
20     def generate(self, input_ids:List[int],
21                 tag_name:str="default"):
22         return self.model.process(prompt)
23
24 # 3. Define a serverless reward computation func
25 class ServerlessRewardWorker(RewardCls):
26     @rdist.register_serverless(
27         attribute='reward_proxy',
28         serverless_url='fc://xxx.xxx')
29     def compute_rewards(self, traj: list):
30         prompt = f"Evaluate the trajectory:{traj}"
31         return ray.get(self.reward_proxy(prompt))
```

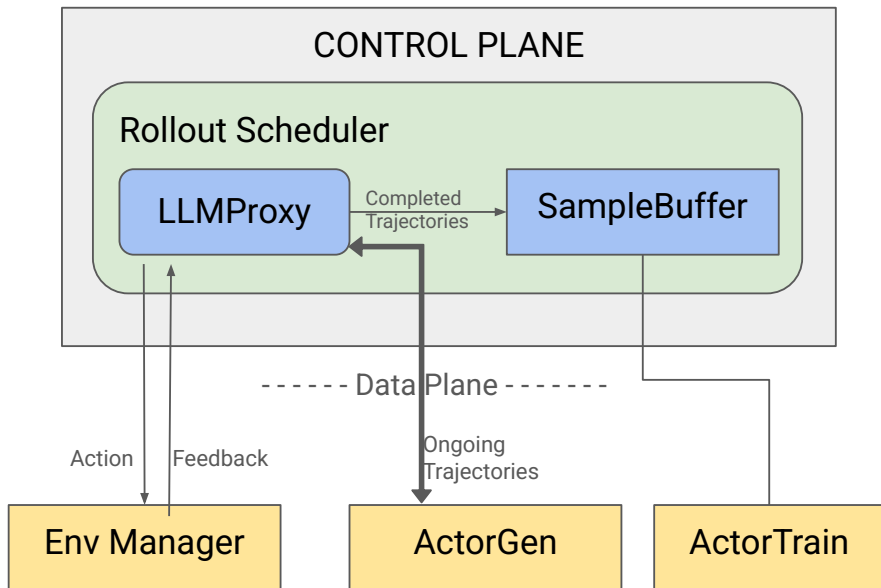
System-Managed Control Plane

Topmost in ROLLART sits Control Plane - entirely system managed (no user-specified coordination).

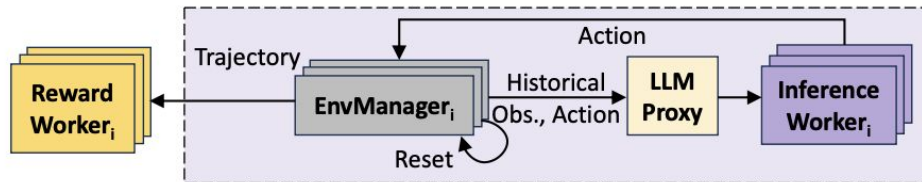
Focus: Orchestration logic, state tracking, and feedback loop management that prevents GPU idling and training divergence.

Core Components

1. **LLMProxy** - Controls ongoing trajectory generation requests and acts as the central router.
2. **SampleBuffer** - Receives completed trajectories and holds versioned batches for the ActorTrain Cluster.



Trajectory-Level Rollout and Reward Overview

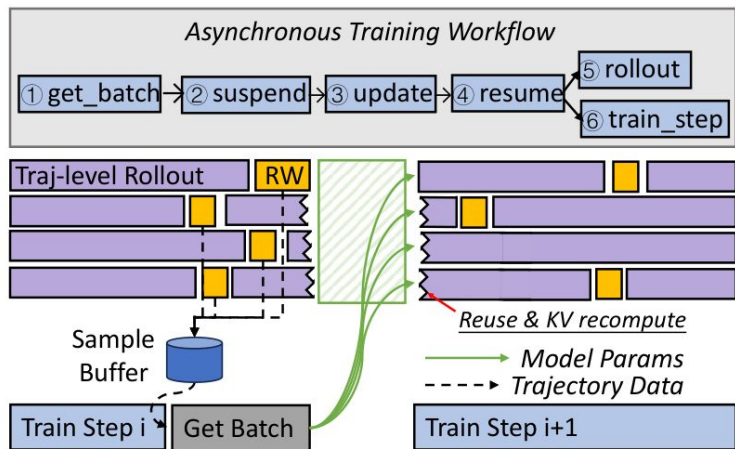


ROLLART decouples trajectory generation, environment interaction, and reward evaluation into independent, asynchronous streams.

Figure 8: Trajectory-Level Rollout and Reward Overview.

- **Pipeline Level**
Eliminates strict barriers between generation, training, and evaluation.
- **Trajectory Level**
Streams rollouts individually instead of waiting for full batch completion.
- **Model Level**
Prevents worker blocking using in-flight KV-cache recomputation.
- **Cluster Level**
Disaggregates workloads across heterogeneous, non-interconnected hardware.

Asynchronous Training Protocol & Staleness Bounding (α)



The Staleness Challenge

- Asynchronous rollouts yield trajectories generated on older model parameters.
- Unbounded staleness causes policy drift and training instability.

RollART's Solution: Bounded Asynchrony (α)

- Enforces a hard threshold α : discards trajectories generated on models older than version $n - \alpha$.

In-Flight KV-Cache Recomputation

- Recomputes KV caches dynamically when new weights arrive during active rollouts (Steps 5 & 6), avoiding trajectory aborts.

Cross-Cutting System Optimizations

Hardware-Specialized Serving

Matches workloads to optimal hardware (H20 for decode, H800 for prefill).

Bucketized Weight Streaming

Streams model weights in asynchronous background chunks via Mooncake to hide latency

Redundant Rollouts

Over-provisions environments to mitigate stragglers so slow rollouts don't block model updates.

Evaluation - What is Being Tested?

End-to-End Speedup

How much higher throughput does trajectory-level decoupling achieve compared to synchronous baselines (e.g. AReaL) ?

Staleness Bounding (α)

How does varying the staleness threshold α impact training convergence and generation quality?

System Scalability

How efficiently does RollART scale across heterogeneous GPU clusters and large model sizes?

Ablation Studies

What are the individual performance contributions of bucketized weight streaming and in-flight KV recomputation?

Evaluation Setup

Infrastructure

96x H800 + 32x H20
400 Gbps InfiniBand
200 Gbps Ethernet
CPU + Serverless
Mooncake v0.3.7

Models

Qwen3 (8B,14B,32B)
SWE-Bench (32B)
Qwen2.5-7B (Reward)

Training and Engine

GRPO
32 Training GPUs
96 Rollout GPUs
vLLM v0.8.4
NCCL v2.26.5

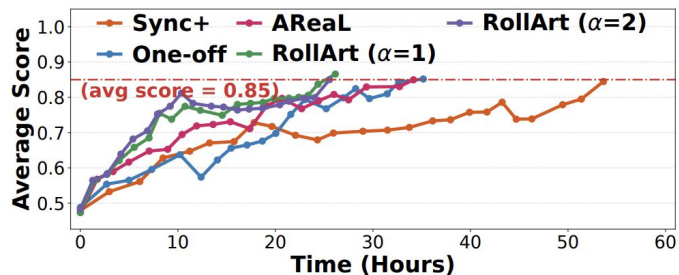
Baselines

Sync & Sync+ (RL)
One-off (Stale iteration)
AReal (bound = 1)
*RollART runs at ~83%
per-GPU-hour cost*

Throughput: Total prompt + response tokens divided by average step time (over 5 iterations).

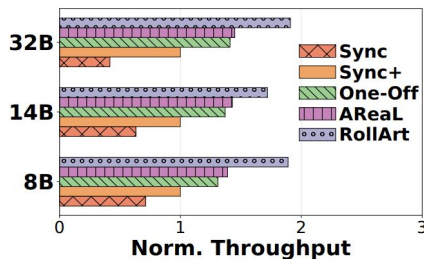
This setup tests whether RollART's asynchronous coordination scales across heterogeneous GPU clusters (H800 + H20) and complex multi-turn agentic workloads without policy degradation.

Quantitative Evaluation & Production Deployment



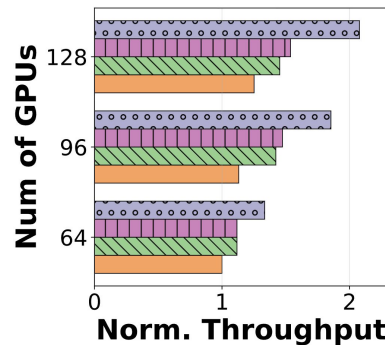
(a) Time-to-Score (target 0.85) on Qwen3-32B.

>2.5x Faster



(b) Norm. Throughput across LLMs.

~2x Throughput



(c) Scaling Efficiency.

Near-Linear Scaling

Production Deployment: Deployed on Alibaba's Qoder agentic platform, running stably across >3,000 GPUs.

R3: Serverless Offloading

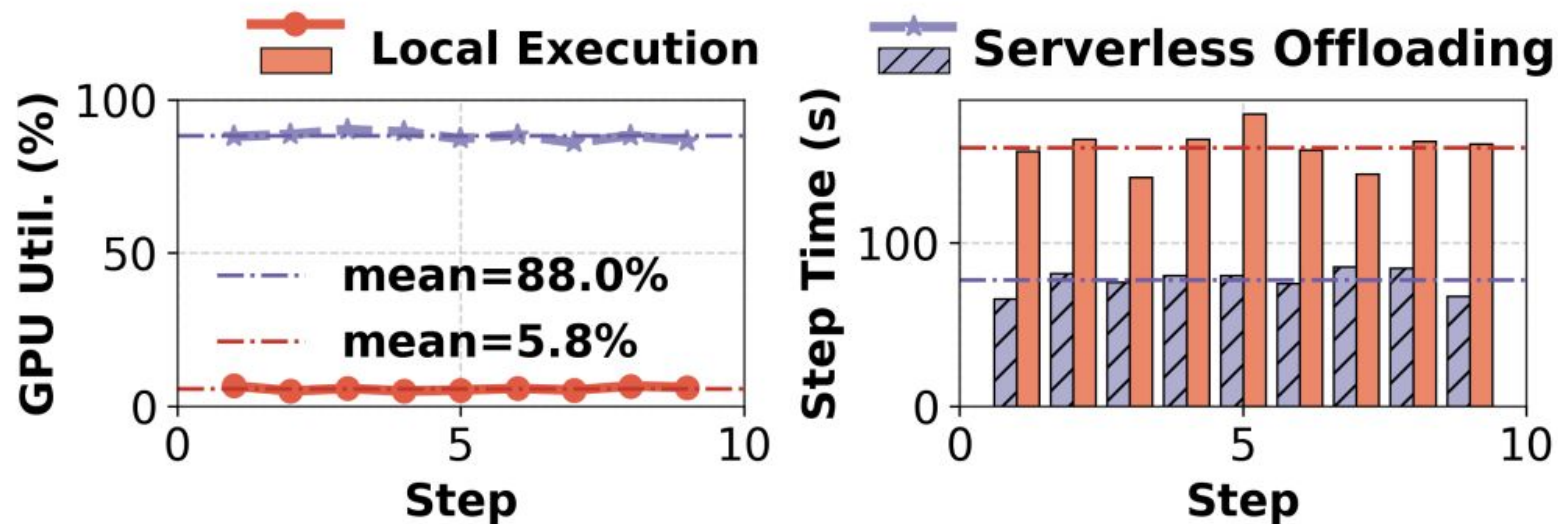


Figure 12: Comparison between dedicating local GPUs and using serverless offloading.

System Strengths & Limitations

System Strengths

Zero GPU Idle Time

Trajectory-level decoupling and asynchronous weight streaming

Bounded Asynchrony (α)

Prevents policy divergence while delivering throughput gains (2.65x – 4.58x)

Industrial Scalability

Deployed across 3,000+ heterogeneous GPUs powering Alibaba's Qoder product

Limitations

Data Synchronization & Memory Overhead

Cross-cluster weight streaming and recomputation creates transient memory spikes

Cloud & Serverless Dependencies

Offloading rewards requires elastic serverless infra, introducing cold-start latency in sandboxed envs

Staleness Bounds

High staleness ($\alpha \geq 3$) can cause policy drift

Future Work & Architectural Extensions

Adaptive Staleness Control

- *Current*: Fixed, static staleness bound (α) set before training.
- *Future*: Dynamically adjusts α based on live gradient variance and policy drift.

Multi-Modal Environment Sandboxes

- *Current*: Optimized primarily for text-based code and math environments.
- *Future*: Expands serverless execution to vision, GUI, and robotics simulations.

Auto-Scaling across GPU Clusters

- *Current*: Fixed GPU allocation between training (**ActorTrain**) and generation (**ActorGen**).
- *Future*: Dynamically shifts GPUs between planes based on real-time request queues.

Fault-Tolerant Dynamic Worker Re-Balancing

- *Current*: Node failures require manual intervention or full job restarts.
- *Future*: Automatically migrates **SampleBuffer** state away from failing nodes during 3,000+ GPU runs.

Summary: RollArt vs. HybridFlow

HybridFlow (EuroSys '25)

RollArt (OSDI '26)

Problem scope

Single/few-turn RLHF alignment (chat-style)

Multi-turn agentic RL with environments (tool use, web, code)

Hardware model

Homogeneous GPU cluster; optimizes placement & parallelism

Heterogeneous, disaggregated: compute-GPUs + bandwidth-GPUs + CPU + serverless

Concurrency granularity

Stage-level dataflow — actor reshards between train/gen on the same devices

Trajectory-level async — rollout, reward, training overlap with bounded staleness (α)

Environment & reward

No environment stage; reward is a GPU forward pass

Environment is a CPU-bound stage; reward offloaded to serverless

Key system technique

3D-HybridEngine: zero-redundancy actor resharding between training & generation

Hardware-affinity task routing + serverless reward + bounded-staleness async training

Thank You